

TECNOLOGÍA EN DESARROLLO DE SISTEMAS INFORMÁTICOS

Internet de las Cosas (IoT) – TSI519

Semana 8

Entradas y Salidas Digitales en ESP32

Sistema de Ventilación Inteligente

JHON SEBASTIAN REYES ROZO

HARVIC EDISSON QUINTERO CACERES

INGENIERO:

ELIECER MONTERO OJEDA

2026

Contenido

Contenido	2
1. Introducción y objetivos	3
1.1. Objetivo general.....	3
1.2. Objetivos específicos.....	3
1.3. Escenario del laboratorio	3
2. Materiales y componentes	3
3. Diseño de la lógica de control	4
3.1. Tabla de automatización del aula	4
3.2. Asignación de pines GPIO.....	4
3.3. Pseudocódigo.....	5
4. Diagrama de flujo.....	6
5. Implementación	7
5.1. Simulación en Wokwi.....	7
5.2. Montaje físico	8
5.3. Código fuente.....	9
6. Pruebas y resultados.....	11
6.1. Evidencias de funcionamiento	11
6.2. Video de demostración	¡Error! Marcador no definido.
6.3. Repositorio.....	13
7. Preguntas de reflexión	14
1. ¿Por qué es importante validar las lecturas del sensor antes de tomar decisiones?	14
2. ¿Qué diferencia hay entre INPUT e INPUT_PULLUP? ¿Cuándo usar cada uno?	14
3. ¿Cómo podríamos agregar histéresis a la lógica de control?	14
4. ¿Qué actuadores adicionales podríamos integrar al proyecto del aula?	14
5. ¿Cómo se relaciona esta semana con el objetivo final del proyecto integrador?	14
8. Conclusiones	14
9. Referencias	15

(Si la tabla aparece vacía: clic derecho > Actualizar campo > Actualizar toda la tabla.)

1. Introducción y objetivos

En las semanas anteriores el sistema de monitoreo del aula solo leía sensores y mostraba los datos. En esta semana el sistema se vuelve activo: toma decisiones a partir de lo que mide y ejecuta acciones sobre el mundo físico mediante actuadores (LEDs, buzzer y relé) controlados por el ESP32.

La pregunta que guía la actividad es: ¿cómo automatizar una respuesta cuando la temperatura del aula supera un valor límite?

1.1. Objetivo general

Controlar actuadores digitales (LEDs, buzzer y relé) a partir de condiciones definidas por sensores, implementando lógica de control con condicionales if/else sobre el ESP32.

1.2. Objetivos específicos

- Comprender el funcionamiento de los pines GPIO del ESP32.
- Diferenciar entre pinMode(), digitalWrite() y digitalRead().
- Implementar lógica de control basada en condiciones de sensores.
- Diseñar la lógica de automatización para el proyecto integrador.

1.3. Escenario del laboratorio

Se construyó un sistema que lee la temperatura con el sensor DHT11, enciende el LED rojo cuando la temperatura es mayor a 28 °C, enciende el LED verde cuando es menor o igual a 28 °C y activa un buzzer como alerta sonora. Adicionalmente se integraron la humedad y la luminosidad como variables de control del aula.

2. Materiales y componentes

Componente	Cant.	Función en el sistema
ESP32 DevKit	1	Microcontrolador; lee sensores y controla actuadores (3.3 V).
Sensor DHT11	1	Mide temperatura y humedad del aula.
LDR + resistencia de 10 kΩ	1	Mide la luminosidad (divisor de tensión, entrada analógica).
LED rojo	1	Indica alerta de temperatura alta.
LED verde	1	Indica estado normal.
LED amarillo	1	Indica alerta de humedad fuera de rango.
LED blanco	1	Simula la iluminación del aula.
Resistencias de 220 Ω	4	Limitan la corriente de cada LED.
Buzzer activo	1	Alerta sonora.

Componente	Cant.	Función en el sistema
Módulo relé de 1 canal	1	Controla el ventilador (lado de control a 5 V desde VIN).
Protoboard y cables dupont	–	Montaje del circuito.
Cable USB	1	Alimentación y programación del ESP32.

Nota: ajusta esta tabla a los componentes que realmente usó tu equipo.

3. Diseño de la lógica de control

3.1. Tabla de automatización del aula

Las cinco primeras filas corresponden a la actividad; las dos últimas son condiciones adicionales propias del contexto del aula.

Variable	Condición	Acción	Actuador
Temperatura	> 28 °C	Encender ventilación y alerta sonora	Relé / LED rojo / Buzzer
Humedad	< 40 %	Generar alerta	Buzzer / LED amarillo
Luminosidad	Oscuridad (LDR bajo)	Encender iluminación	LED blanco / Relé
Temperatura	≤ 28 °C	Estado normal	LED verde
Humedad	≥ 40 %	Estado normal	LED verde
Humedad (propia)	> 70 %	Alerta de humedad alta (riesgo para equipos y confort)	Buzzer / LED amarillo
Luminosidad (propia)	Luz suficiente (LDR alto)	Apagar iluminación para ahorrar energía	LED blanco apagado / Relé

3.2. Asignación de pines GPIO

Los pines D4, D5, D18, D19, D21, D22 y D23 de la placa corresponden a los GPIO con el mismo número.

Componente	Tipo	GPIO	Observación
Sensor DHT11 (datos)	Entrada	4	Lectura digital de temperatura y humedad.
LED rojo	Salida	5	Con resistencia de 220 Ω.
LED verde	Salida	18	Con resistencia de 220 Ω.
Buzzer	Salida	19	Buzzer activo.
Relé (ventilador)	Salida	21	VCC a VIN (5 V), GND común.
LED amarillo	Salida	22	Con resistencia de 220 Ω.

Componente	Tipo	GPIO	Observación
LED blanco	Salida	23	Con resistencia de 220 Ω .
LDR	Entrada analógica	34	Pin solo entrada (ADC).

Se evitaron los GPIO 6 a 11 (conectados a la memoria flash) y se usó el GPIO 34, que es solo de entrada, para el LDR.

3.3. Pseudocódigo

```

INICIO
  Configurar pines: LED_ROJO, LED_VERDE, BUZZER, RELE,
                  LED_AMARILLO, LED_BLANCO como SALIDA
  Configurar LDR como ENTRADA
  Iniciar Serial a 115200 e iniciar el sensor DHT11

REPETIR PARA SIEMPRE
  temperatura <- leer temperatura del DHT11
  humedad      <- leer humedad del DHT11
  luz          <- leer valor analógico del LDR

  SI temperatura o humedad no son números válidos ENTONCES
    Mostrar "Error leyendo DHT11"
    Esperar 2 s
    Volver al inicio del ciclo
  FIN SI

  SI temperatura > 28 ENTONCES
    LED_ROJO <- ENCENDIDO ; LED_VERDE <- APAGADO ; RELE <- ENCENDIDO
  SI NO
    LED_ROJO <- APAGADO ; LED_VERDE <- ENCENDIDO ; RELE <- APAGADO
  FIN SI

  SI humedad < 40 O humedad > 70 ENTONCES
    LED_AMARILLO <- ENCENDIDO
  SI NO
    LED_AMARILLO <- APAGADO
  FIN SI

  SI luz < UMBRAL_OSCURO ENTONCES
    LED_BLANCO <- ENCENDIDO
  SI NO
    LED_BLANCO <- APAGADO
  FIN SI

  SI (temperatura > 28) O (humedad fuera de rango) ENTONCES
    BUZZER <- ENCENDIDO
  SI NO
    BUZZER <- APAGADO
  FIN SI

  Mostrar por Serial: temperatura, humedad, luz y acciones
  Esperar 2 s
FIN REPETIR

```

4. Diagrama de flujo

El diagrama representa la lectura de sensores, las decisiones condicionales (if/else), las acciones sobre los actuadores y los mensajes de depuración por el monitor serial.

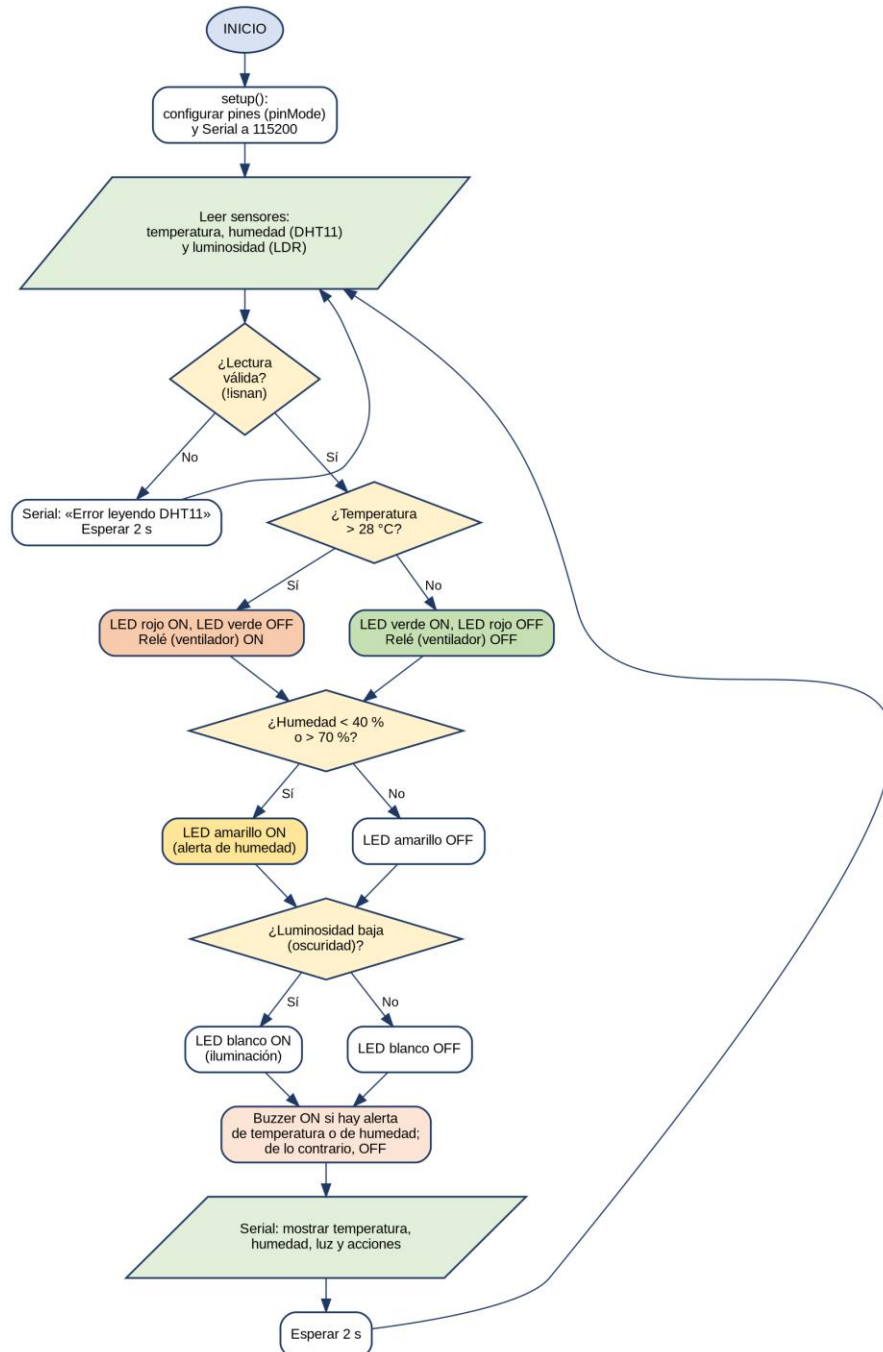


Figura 1. Diagrama de flujo del sistema de automatización.

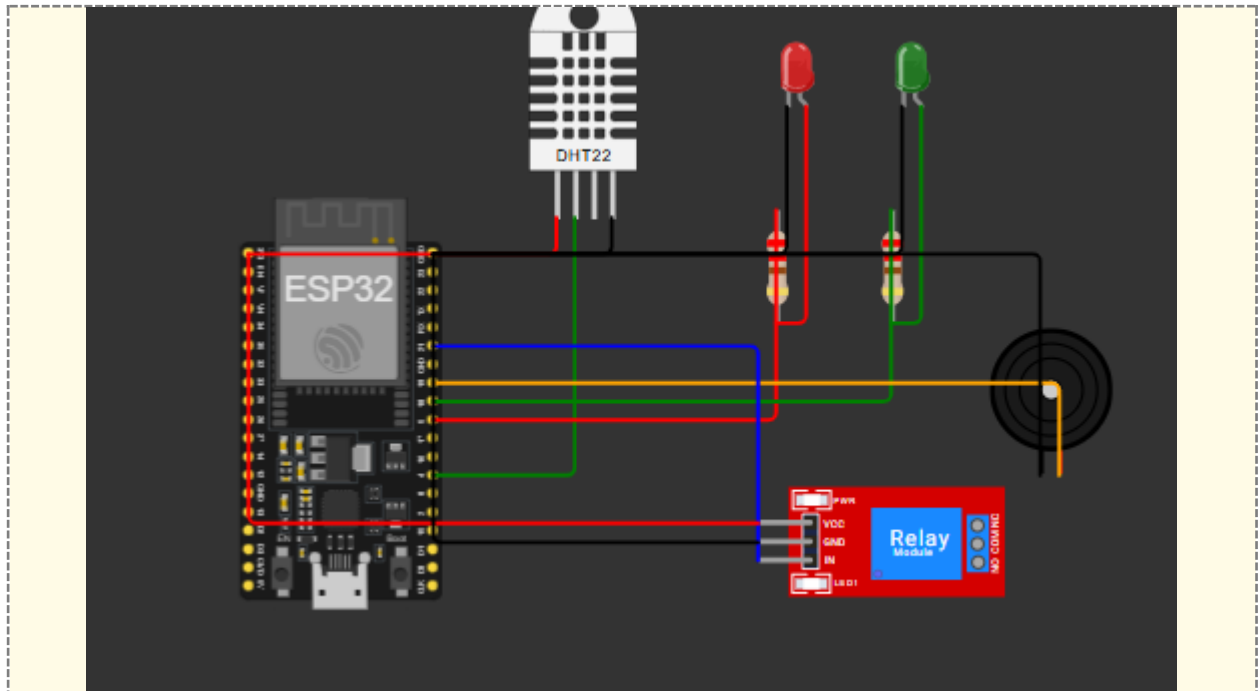
Opcional: si el docente exige el diagrama en Draw.io, puedes rehacerlo allí con la misma estructura y reemplazar esta imagen.

5. Implementación

5.1. Simulación en Wokwi

Enlace al proyecto en Wokwi: <https://wokwi.com/projects/307248101781930560>

Figura 2. Circuito armado en Wokwi



5.2. Montaje físico

Figura 3. Circuito físico armado

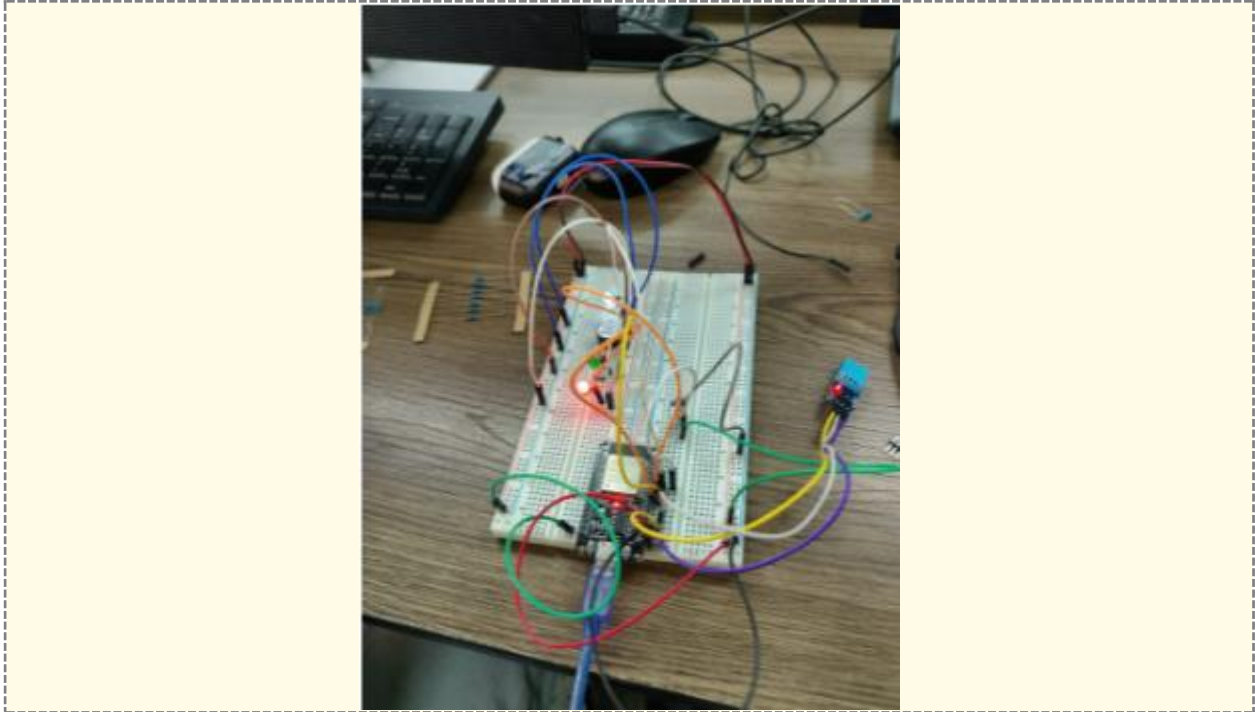
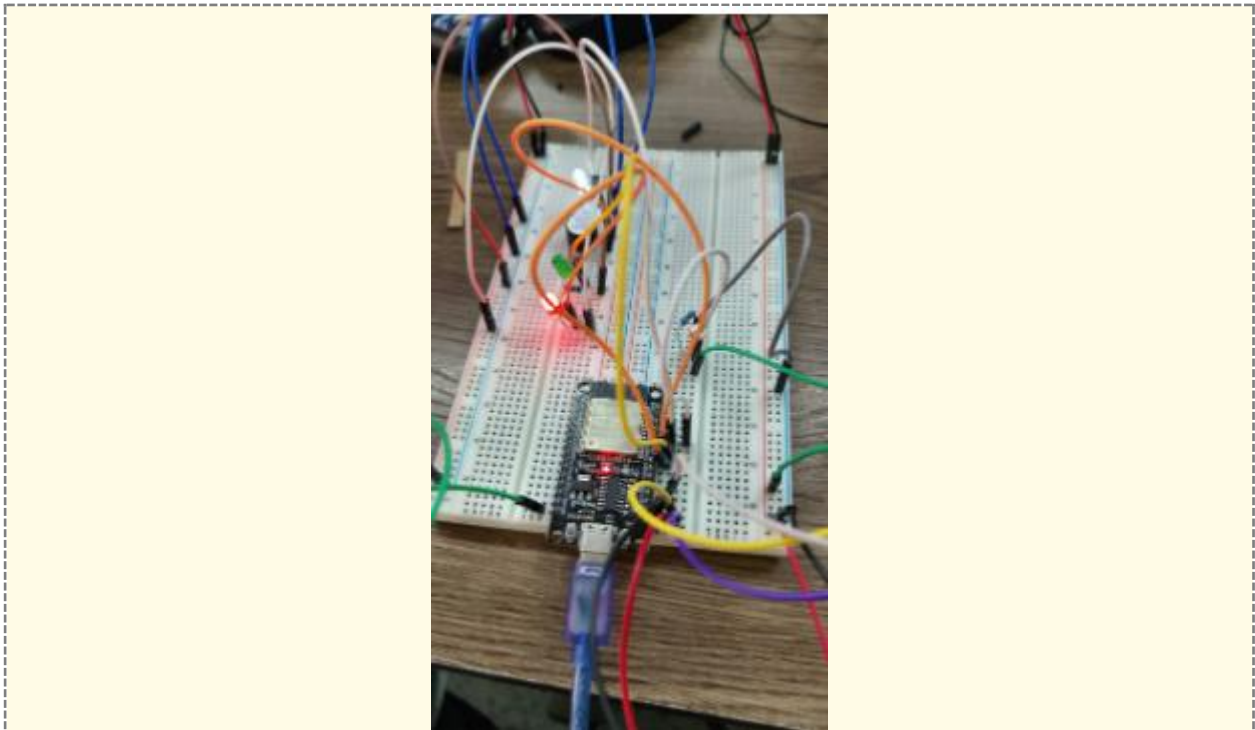


Figura 4. Detalle de conexiones



5.3. Código fuente

El código completo está comentado y se encuentra en el repositorio de GitHub del equipo. Se incluye a continuación:

```
/*
- Poca luz (oscuridad) -> LED blanco (iluminación).
*/
#include <DHT.h>

// ----- Pines (GPIO) -----
#define DHTPIN      4    // DHT11 (datos)
#define LED_ROJO    5    // Alerta de temperatura
#define LED_VERDE   18   // Estado normal
#define BUZZER      19   // Alerta sonora
#define RELE        21   // Ventilador
#define LED_AMARILLO 22  // Alerta de humedad
#define LED_BLANCO  23   // Iluminación
#define LDR_PIN     34   // Solo entrada (ADC)

#define DHTTYPE DHT11    // En Wokwi usar DHT22 si no aparece el DHT11

// ----- Umbrales -----
const float TEMP_LIMITE = 28.0; // °C
const float HUM_MINIMA  = 40.0; // %
const float HUM_MAXIMA  = 70.0; // %
const int   UMBRAL_OSCURO = 1000; // ADC 0-4095 (calibrar según el montaje)

DHT dht(DHTPIN, DHTTYPE);

void setup() {
  Serial.begin(115200);

  // Salidas
  pinMode(LED_ROJO, OUTPUT);
  pinMode(LED_VERDE, OUTPUT);
  pinMode(BUZZER, OUTPUT);
  pinMode(RELE, OUTPUT);
  pinMode(LED_AMARILLO, OUTPUT);
  pinMode(LED_BLANCO, OUTPUT);

  // Entrada
  pinMode(LDR_PIN, INPUT);

  dht.begin();
}

void loop() {
  // 1. Lectura de sensores
  float temperatura = dht.readTemperature();
  float humedad     = dht.readHumidity();
  int   luz         = analogRead(LDR_PIN);

  // 2. Validar lectura antes de decidir
  if (isnan(temperatura) || isnan(humedad)) {
    Serial.println("Error leyendo DHT11");
    delay(2000);
    return;
  }
}
```

```

// 3. Evaluar condiciones
bool tempAlta = (temperatura > TEMP_LIMITE);
bool humAlerta = (humedad < HUM_MINIMA || humedad > HUM_MAXIMA);
bool oscuro = (luz < UMBRAL_OSCURO);

// 4. Actuar sobre los actuadores
digitalWrite(LED_ROJO, tempAlta ? HIGH : LOW);
digitalWrite(LED_VERDE, tempAlta ? LOW : HIGH);
digitalWrite(RELE, tempAlta ? HIGH : LOW);
digitalWrite(LED_AMARILLO, humAlerta ? HIGH : LOW);
digitalWrite(LED_BLANCO, oscuro ? HIGH : LOW);
digitalWrite(BUZZER, (tempAlta || humAlerta) ? HIGH : LOW);

// 5. Mensajes de depuración
Serial.print("Temp: "); Serial.print(temperatura); Serial.print(" C | ");
Serial.print("Hum: "); Serial.print(humedad); Serial.print(" % | ");
Serial.print("Luz: "); Serial.println(luz);
Serial.println(tempAlta ? "ALERTA: temperatura alta - ventilación ON" : "Temperatura normal");
Serial.println(humAlerta ? "ALERTA: humedad fuera de rango" : "Humedad normal");
Serial.println(oscuro ? "Poca luz - iluminación ON" : "Luz suficiente");
Serial.println("-----");

delay(2000);
}

```

Funciones clave utilizadas:

- `pinMode(pin, modo)`: configura cada pin como entrada o salida (una sola vez en `setup()`).
- `digitalWrite(pin, valor)`: enciende (HIGH, 3.3 V) o apaga (LOW, 0 V) cada actuador.
- `digitalRead(pin)`: lee el estado de una entrada digital (por ejemplo, un pulsador).
- `isnan()`: valida que la lectura del DHT11 sea correcta antes de tomar decisiones.
- `if / else`: decide qué actuadores activar según los umbrales definidos.

6. Pruebas y resultados

Se probó el sistema variando las condiciones del ambiente. Completa la columna «Resultado observado» con lo que viste en tu montaje.

Condición probada	Resultado esperado	Actuadores
Temperatura $\leq 28^{\circ}\text{C}$	Estado normal	LED verde ON; rojo, buzzer y relé OFF
Temperatura $> 28^{\circ}\text{C}$	Alerta y ventilación	LED rojo ON, buzzer ON, relé ON
Humedad $< 40\%$	Alerta de humedad baja	LED amarillo ON, buzzer ON
Humedad $> 70\%$	Alerta de humedad alta	LED amarillo ON, buzzer ON
Poca luz (LDR bajo)	Iluminación encendida	LED blanco ON
Sensor desconectado	Mensaje de error	Serial: «Error leyendo DHT11»

6.1. Evidencias de funcionamiento

Figura 5. Estado normal (temperatura $\leq 28^{\circ}\text{C}$)

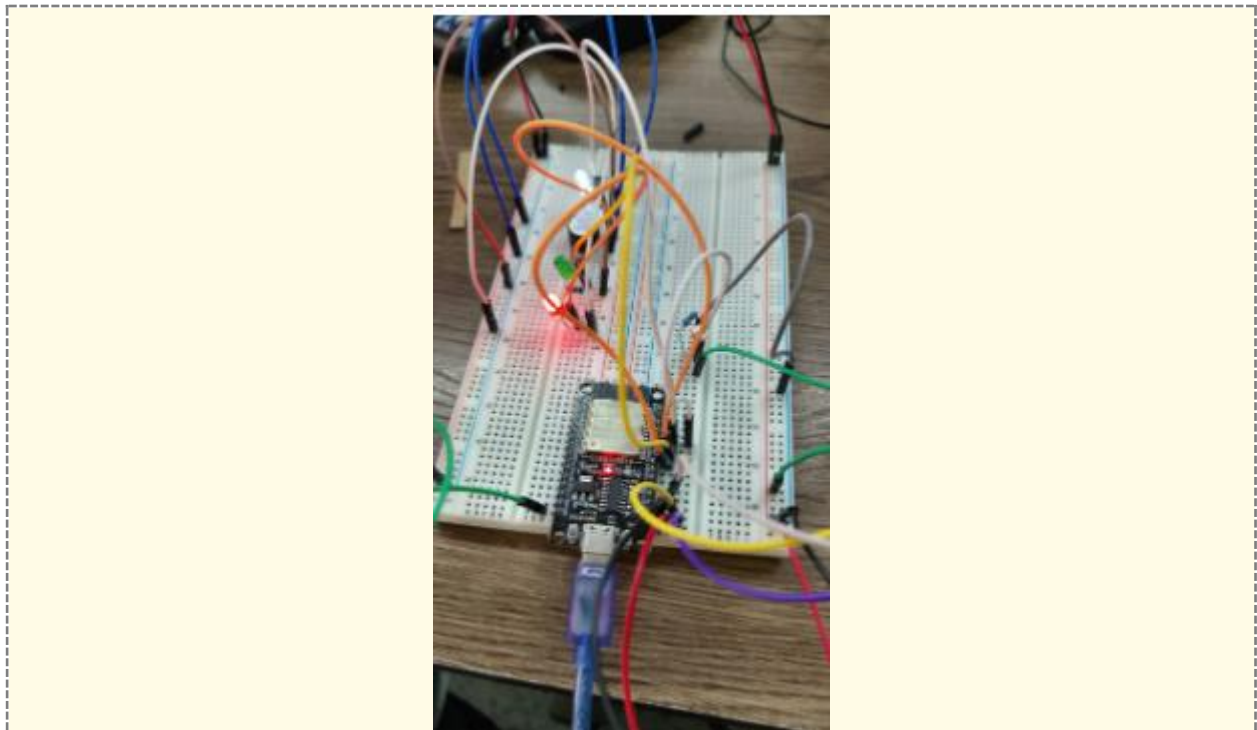


Figura 6. Estado de alerta (temperatura > 28 °C)

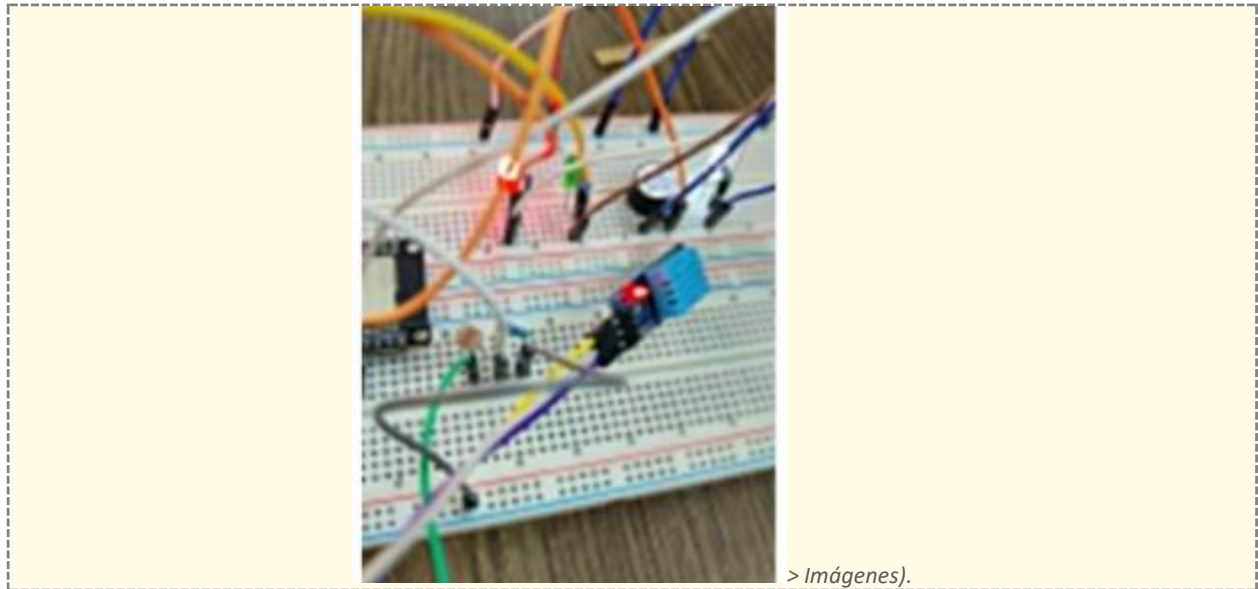


Figura 7. Monitor serial – estado normal

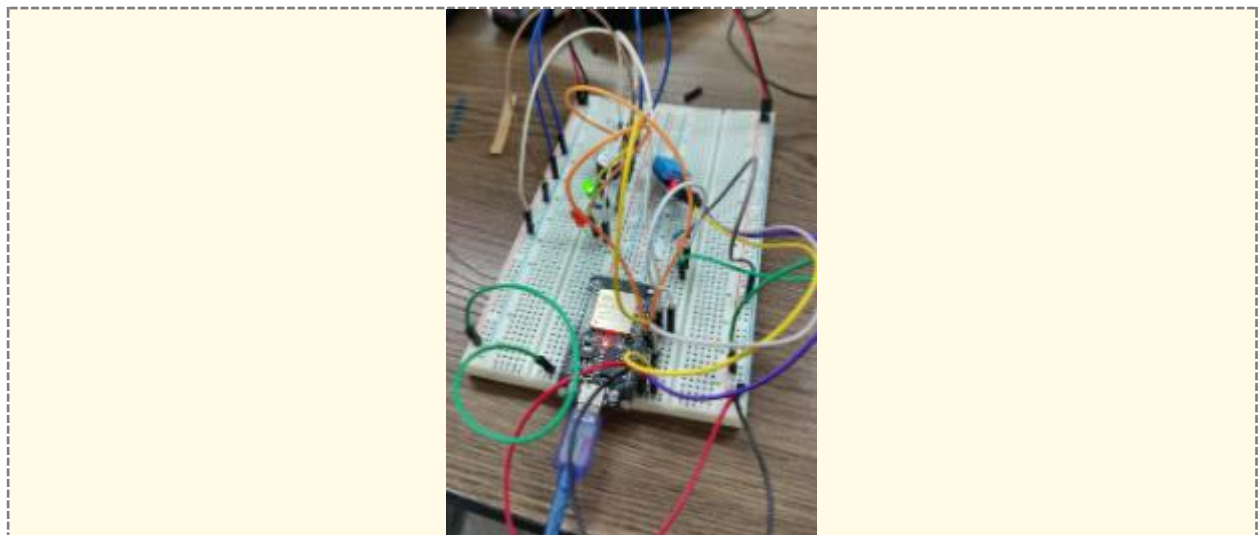


Figura 8. Monitor serial – estado de alerta

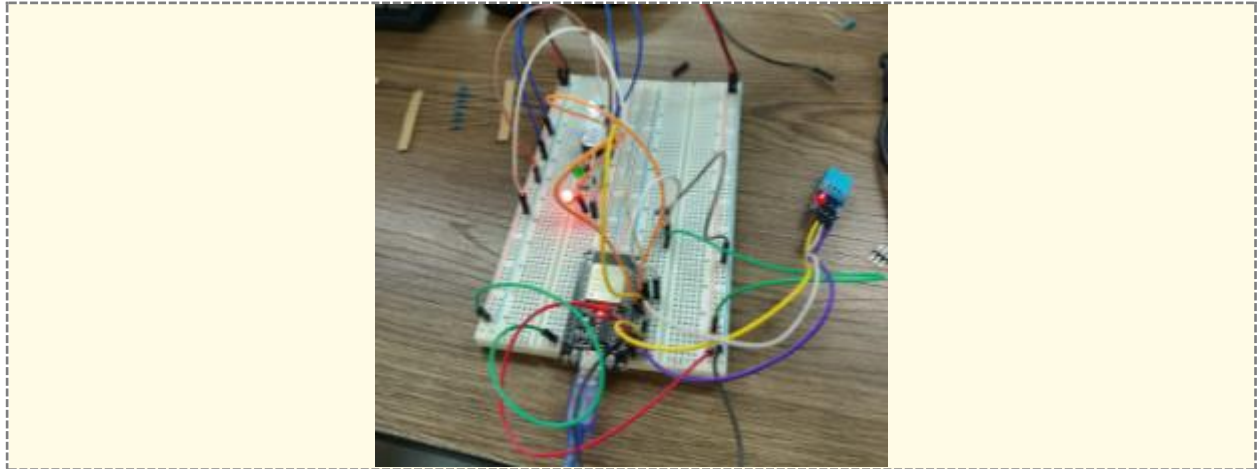
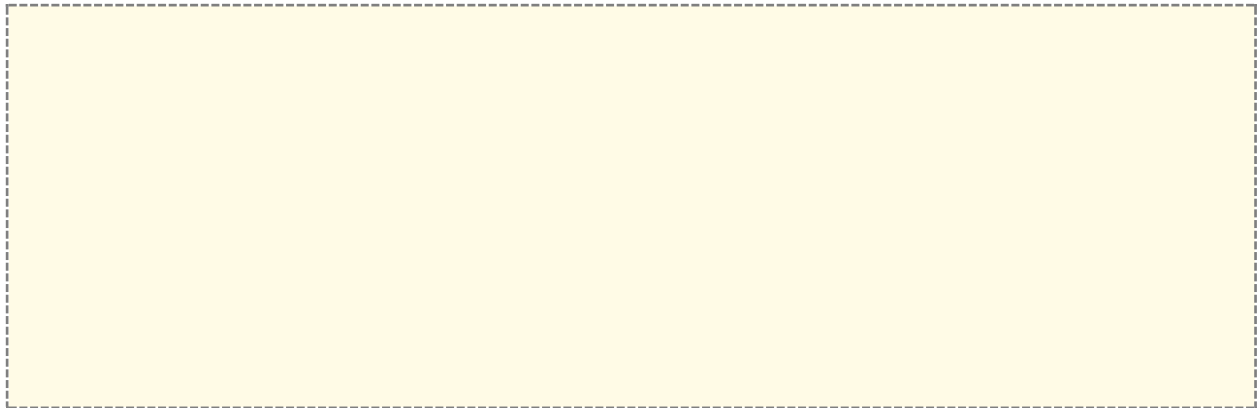


Figura 9. Alertas de humedad y luminosidad



6.3. Repositorio

Enlace al repositorio de GitHub: https://github.com/HQED/loT_Proyecto

7. Preguntas de reflexión

Las respuestas son un borrador; revísalas y ajústalas con las palabras de tu equipo.

1. ¿Por qué es importante validar las lecturas del sensor antes de tomar decisiones?

Porque un sensor puede fallar o devolver datos inválidos (por ejemplo NaN si el DHT11 está desconectado o hay ruido). Si el sistema actúa con un dato erróneo, podría encender o apagar actuadores sin motivo o no reaccionar ante una situación real. Por eso se usa `isnan()` y se descarta la lectura antes de aplicar el `if/else`.

2. ¿Qué diferencia hay entre INPUT e INPUT_PULLUP? ¿Cuándo usar cada uno?

INPUT deja el pin flotante: lee lo que le llegue desde fuera y necesita una resistencia externa si el estado no está definido. INPUT_PULLUP activa una resistencia interna conectada a 3.3 V, de modo que el pin lee HIGH por defecto y LOW cuando se cierra a GND. Se usa INPUT con sensores que ya entregan un nivel definido y INPUT_PULLUP con pulsadores o interruptores, ahorrando la resistencia externa (la lógica queda invertida: presionado = LOW).

3. ¿Cómo podríamos agregar histéresis a la lógica de control?

Usando dos umbrales en lugar de uno. Por ejemplo, la alerta se activa cuando la temperatura supera 28 °C y solo se desactiva cuando baja de 26 °C. Se guarda el estado en una variable booleana y solo cambia al cruzar el umbral correspondiente. Así se evita que el relé y el buzzer se enciendan y apaguen rápidamente cuando la temperatura oscila cerca de 28 °C.

4. ¿Qué actuadores adicionales podríamos integrar al proyecto del aula?

- Pantalla LCD u OLED para mostrar temperatura y humedad en el aula.
- Servomotor para abrir y cerrar ventanas o persianas automáticamente.
- Tira LED o lámparas controladas por relé según la luminosidad.
- Aire acondicionado o ventilador de mayor potencia mediante relé de estado sólido.
- Notificaciones a la nube (Blynk o Adafruit IO) para avisar al docente.

5. ¿Cómo se relaciona esta semana con el objetivo final del proyecto integrador?

El proyecto busca un sistema IoT de monitoreo inteligente del aula. Hasta la Semana 7 el sistema solo leía datos; con esta semana incorpora la parte de decisión y acción (automatización local). En las próximas semanas estos datos y acciones se enviarán a la nube por WiFi (Semana 9), MQTT (Semana 10) y Blynk (Semana 12), completando el ciclo sensor – decidir – actuar – monitorear a distancia.

8. Conclusiones

- Se logró controlar actuadores digitales (LEDs, buzzer y relé) a partir de las lecturas del DHT11 y del LDR usando condicionales `if/else` en el ESP32.
- La validación de las lecturas con `isnan()` evita decisiones basadas en datos erróneos.
- Es fundamental respetar los 3.3 V del ESP32, usar resistencias en los LEDs y evitar los pines reservados para la memoria flash.

- La lógica diseñada es la base para integrar la comunicación con la nube en las siguientes semanas.

9. Referencias

Adafruit Learning System. (2024). Adafruit Learning System.

Arduino. (2023). Arduino Documentation – Digital Pins.

Blynk Documentation. (2024).

Casado-Vara, R., & Prieto, J. (2020). IoT Fundamentals with ESP32.

Espressif Systems. (2024). ESP32 Programming Guide.

Knight, N. (2022). Learn Electronics with Arduino.

Wokwi. (s. f.). Wokwi – Online ESP32 Simulator. <https://wokwi.com>

Adafruit. (s. f.). DHT sensor library. <https://github.com/adafruit/DHT-sensor-library>